KB004 - API Gateway and Microservice Failure Troubleshooting Guide

---

Document Metadata

Field	Value
Document ID	KB004
Document Type	Production Support Runbook / Known Error Document
Service Area	API Gateway and Microservices Platform
Application Owner	Platform Engineering Team
Support Group	Platform SRE L2
Severity Coverage	P1, P2, P3
Environment	Production, Staging
Last Updated	2026-02-01
Review Cycle	Quarterly

---

1. Overview

The API Gateway layer acts as the entry point between client applications and backend microservices.

It is responsible for:

- Request routing
- Authentication validation
- Rate limiting
- Traffic management
- Service discovery
- Request transformation
- Load balancing

API Gateway incidents usually appear as customer-facing failures but are frequently caused by downstream dependency issues.

Affected systems may include:

- Payment Gateway

- Authentication Service
- Order Service
- Customer Profile Service
- Notification Service

Common business impacts:

- Users unable to access application features
- API requests failing
- Increased application latency
- Partial service outages
- Increased customer complaints

---

2. Common Incident Symptoms

Customer Symptoms

Users may report:

- Application pages not loading
- Payment requests failing
- Login requests timing out
- Mobile application errors
- Slow response times

API Symptoms

Common HTTP errors:

```
HTTP 500 Internal Server Error

HTTP 502 Bad Gateway

HTTP 503 Service Unavailable

HTTP 504 Gateway Timeout

HTTP 429 Too Many Requests
```

Example API Gateway logs:

```
ERROR GatewayRouter

Request forwarding failed.
```

```
Exception:

UpstreamServiceUnavailableException

Message:

Unable to connect to downstream service
```

Another common error:

```
GatewayTimeoutException:

Response not received from backend service within timeout window.
```

---

3. Monitoring Indicators

Verify API Gateway health metrics.

Gateway Metrics

Metric	Warning Threshold	Critical Threshold
API Error Rate	>5%	>15%
P95 Latency	>2 seconds	>5 seconds
Request Queue Size	>1000	>5000
CPU Usage	>75%	>90%
Memory Usage	>80%	>95%

Important metrics:

```
api.request.count

api.error.rate

api.response.time

gateway.timeout.count

upstream.failure.count
```

---

4. Common Root Causes and Resolution Steps

RCA Pattern 1: Downstream Microservice Failure

Description

The API Gateway routes requests to multiple backend services.

If a downstream service becomes unhealthy, gateway failures occur even though the gateway itself is operational.

Examples:

Payment API request flow:

```
Client

  |
  v

API Gateway

  |
  v

Payment Service

  |
  v

Database
```

Failure in Payment Service or Database can appear as API Gateway timeout.

Symptoms

Gateway logs:

```
HTTP 502 Bad Gateway

upstream connection refused
```

or:

```
HTTP 504 Gateway Timeout

backend response timeout exceeded
```

Monitoring:

```
backend_health_status = unhealthy

dependency_error_rate increasing
```

Validation Steps

Check backend service health:

```
curl /health/payment-service
```

Expected:

```
STATUS: HEALTHY
```

Check service logs:

```
grep ERROR application.log
```

Validate dependency:

```
gateway_latency

vs

backend_latency
```

Immediate Resolution

Actions:

1. Restart unhealthy microservice instances.
2. Remove unhealthy nodes from load balancer.
3. Redirect traffic to healthy region.
4. Scale backend service.

Example:

```
scale payment-service replicas=10
```

Permanent Fix

Implement:

- Better health checks
- Automatic failover
- Circuit breaker patterns
- Dependency monitoring

RCA Pattern 2: Traffic Spike / Capacity Exhaustion

Description

Sudden traffic growth can overload API Gateway or backend services.

Common causes:

- Marketing campaigns
- Seasonal sales

- Unexpected user growth
- Automated bot traffic

Symptoms

Metrics:

```
CPU usage > 90%

request queue increasing

thread pool exhausted
```

Application logs:

```
RejectedExecutionException

No worker threads available
```

Validation

Compare:

Current request rate:

```
50,000 requests/min
```

Normal baseline:

```
10,000 requests/min
```

Check autoscaling events:

```
replica_count

autoscaling_activity
```

Immediate Resolution

Actions:

1. Increase gateway replicas.
2. Enable emergency scaling.
3. Apply temporary rate limits.
4. Block abnormal traffic sources.

Example:

```
increase gateway replicas=20
```

Permanent Fix

Implement:

- Predictive scaling
- Load testing
- Capacity planning
- Improved autoscaling rules

RCA Pattern 3: Memory Leak in Microservice

Description

Memory leaks gradually consume application resources until the service becomes unstable.

Common causes:

- Object references not released
- Large cache growth
- Improper resource cleanup

Symptoms

Metrics:

```
memory_usage continuously increasing

garbage_collection_time increasing

application_restart_count increasing
```

Logs:

```
OutOfMemoryError:

Java heap space exceeded
```

Validation

Check memory trends:

```
memory_usage_last_24_hours
```

Capture heap dump:

```
jmap dump application_process_id
```

Immediate Resolution

Actions:

1. Restart affected instances.
2. Increase memory allocation temporarily.
3. Redirect traffic.

Permanent Fix

Development team should:

- Analyze heap dump
- Fix memory leak
- Optimize object lifecycle

RCA Pattern 4: API Rate Limit Configuration Issue

Description

Incorrect gateway policies can block valid customer traffic.

Examples:

- Incorrect rate limits
- Wrong API quotas
- Misconfigured routing rules

Symptoms

Customers receive:

```
HTTP 429 Too Many Requests
```

Gateway logs:

```
Request rejected by rate limiting policy
```

Validation

Check gateway configuration:

```
rateLimit:

requestsPerMinute: 1000
```

Compare:

```
current traffic

vs

allowed threshold
```

Resolution

Immediate:

- Increase allowed request limit
- Restore previous gateway configuration

Permanent:

- Add configuration validation pipeline
- Improve release testing

RCA Pattern 5: Network Connectivity Issue

Description

Network failures between gateway and backend services can interrupt communication.

Possible causes:

- DNS failures
- Firewall changes
- Load balancer problems
- Service discovery issues

Symptoms

Logs:

```
Connection refused

Connection timeout
```

```
DNS resolution failed
```

Validation

Check DNS:

```
nslookup payment-service
```

Test connectivity:

```
curl payment-service/health
```

Resolution

Actions:

- Restore network configuration
- Restart service discovery components
- Update routing rules

---

5. Incident Investigation Checklist

Step 1: Determine Failure Location

Identify:

```
Client issue?

Gateway issue?

Backend issue?

Database issue?
```

Step 2: Review Recent Changes

Check:

- Application deployments
- Gateway configuration updates
- Infrastructure changes
- Network changes

Step 3: Validate Dependencies

Check:

- Authentication Service
- Payment Service
- Database
- External APIs

Step 4: Recover Service

Possible mitigations:

- Restart unhealthy service
- Rollback deployment
- Increase capacity
- Route traffic away

---

6. Escalation Matrix

Issue Type	Escalation Team
Gateway Configuration	Platform Engineering
Backend Failure	Application Team
Database Dependency	Database Engineering
Infrastructure Issue	Cloud Operations
Network Issue	Network Engineering

---

7. Related Historical Incidents

INC-40112

Issue:

Customers received HTTP 504 errors during checkout.

Root Cause:

Payment Service became unavailable behind API Gateway.

Resolution:

Restarted Payment Service instances and increased replicas.

INC-40551

Issue:

API latency increased during promotional event.

Root Cause:

Gateway capacity exhausted due to traffic spike.

Resolution:

Scaled gateway replicas and adjusted autoscaling policy.

INC-40987

Issue:

Multiple APIs returning HTTP 502 errors.

Root Cause:

Incorrect gateway routing configuration after deployment.

Resolution:

Rolled back API Gateway configuration.

---

8. RCA Generation Template

API incident RCA should contain:

Root Cause

Technical component responsible for failure.

Evidence

Include:

- Gateway logs
- Service metrics
- Dependency health
- Historical incidents

Resolution Applied

Immediate recovery actions.

Preventive Actions

Reliability improvements.

Example:

Root Cause:

API Gateway timeout occurred because downstream Payment Service instances became unhealthy.

Evidence:

- Gateway returned HTTP 504 responses.
- Backend health checks failed.
- Similar incident INC-40112 identified.

Resolution:

Restarted Payment Service and increased replicas.

Preventive Action:

Added improved health checks and autoscaling rules.

End of Document